

Framework Comparison for Virtual Try-On and AR Development

The objective of this task was to explore and compare different frameworks and SDKs that can be used for Virtual Try-On (VTON), augmented reality applications, avatar creation, and real-time rendering. The comparison was done based on features, ease of use, performance, and cost.

Tool	Features	Ease of Use	Performance	Cost
<u>Unity</u>	Supports AR/VR, real-time rendering, cross-platform development, large plugin ecosystem	Easy to Moderate	High	Free for personal use, paid enterprise plans
<u>Unreal Engine</u>	Advanced graphics, realistic rendering, simulations, game development	Moderate to Difficult	Very High	Free with royalty-based model
<u>Ready Player Me</u>	3D avatar creation, customization, avatar integration APIs	Easy	Moderate	Free and premium plans
<u>Banuba SDK</u>	Face tracking, beauty filters, virtual try-on support, AR effects	Moderate	High	Paid licensing
<u>DeepAR</u>	Real-time face tracking, AR filters, facial effects	Easy to Moderate	High	Trial available, paid plans
<u>Snap AR</u>	Lens Studio, face/body tracking, AR filters and interactive experiences	Easy	Good	Mostly free

WHY COMPARE DIFFERENT FRAMEWORKS?

Before developing a Virtual Try-On (VTON) system, it is important to understand which technologies are suitable for the required functionality. Different frameworks provide different capabilities such as real-time rendering, face tracking, avatar generation, and AR support. Comparing them helps in selecting the right tool based on project goals, ease of development, system performance, and user experience.

WHAT I DID:

1. Studied different frameworks and SDKs used in AR, virtual try-on systems, and avatar generation.
2. Compared each framework based on features, ease of implementation, system performance, and pricing.
3. Understood which tools are more suitable for building VTON-related applications and real-time user experiences.
4. Prepared a comparison table to better understand the strengths and limitations of each framework.

OBSERVATIONS:

Most Suitable Frameworks for VTON

- After comparison, Banuba SDK, DeepAR, Unity, and Snap AR were found to be more suitable for Virtual Try-On (VTON) applications and real-time user experiences. These tools support features such as augmented reality, face and body tracking, real-time rendering, and interactive effects, which are important for virtual try-on systems. Banuba SDK and DeepAR are useful for real-time tracking and filters, while Unity and Snap AR help in creating interactive and engaging applications.
- These frameworks are more practical for VTON because they can process camera input, track user movement, and display virtual elements smoothly in real time. Their focus on interaction and responsiveness makes them suitable choices for building virtual try-on systems.

Least Suitable Frameworks for VTON

- Ready Player Me and Unreal Engine were comparatively less suitable for direct VTON implementation. Ready Player Me mainly focuses on avatar creation and customization rather than real-time try-on functionality. It can be useful when personalized avatars are required but is not designed specifically for VTON.

- Unreal Engine provides powerful graphics and rendering but is more commonly used for gaming, simulations, and large-scale visual experiences. Although it can support VTON, it may be more complex and resource-heavy compared to frameworks focused mainly on AR and tracking.

CONCLUSION:

This task helped me understand that not every framework serves the same purpose. Some technologies are better for building complete applications, while others are designed mainly for tracking, filters, or avatar generation. I also learned that choosing the right framework depends on factors such as ease of use, performance, project requirements, and the type of user experience expected.